🔑 Answer Key & Full Explanations — Analytics-DA-201 Practice Exam

How to score yourself

Rule	Detail
Pass threshold	**65% → 39 of 60 correct**
Multi-select & build-list items	**No partial credit** — the item counts only if every selected part/order is exactly right
Recommended workflow	Take the paper timed (105 min), mark it with the answer sheet, then study every question you missed + the "Key point" lines below

Question map by domain (report your per-domain score using this)

Domain	Questions	Focus
1. Connect to & Transform Data	Q1 – Q15	connections, joins/relationships/unions, Prep, filters, extracts
2. Explore & Analyze Data	Q16 – Q42	**LODs, calculations, table calcs, filters, parameters, sets, maps**
3. Create Content	Q43 – Q55	charts, dashboards, actions, formatting
4. Publish & Manage on Server/Cloud	Q56 – Q60	publishing, refresh, alerts, custom views, site roles

---

Section 1 — Connect to and Transform Data

Q1. ✅ **Answer: B** — A live connection always reads the database directly, so the dashboard reflects every-minute updates. A, C: extracts are snapshots — even hourly/incremental refreshes cannot show per-minute changes. D: a data source filter has no effect on freshness.

> **Key point:** "Real time / always current / changes every second" → **live connection**. For exam purposes, live is the *only* option that guarantees real-time data.

Q2. ✅ **Answer: D** — Slow database + data changing once per day is the textbook case for an **extract with a scheduled refresh** run after the nightly load: users query Hyper's columnar store locally, interactions become instant. A: live keeps the slow round-trips. B: custom SQL does not fix source latency. C: blending addresses *combining* sources, not speed.

> **Key point:** "Slow source + periodic updates + performance/comportability" → **extract + scheduled refresh**.

Q3. ✅ **Answer: C** — Published data sources from a Tableau Server / Tableau Cloud site are reached through the Connect pane: **To a Server → Tableau Server / Tableau Cloud**, then search published data sources (or *File → Open from Tableau Server/Cloud*). A: the Analytics pane holds analytics objects (reference lines, trends), not data. B: opens local workbooks. D: pastes copied data into a view.

> **Key point:** Published data sources live on Server/Cloud → connect via the **Tableau Server/Tableau Cloud connector** and search.

Q4. ✅ **Answer: B** — **Data → [current data source] → Replace Data Source...** swaps the source for every worksheet that uses it; because field names match, all existing charts keep working. A is manual and error-

prone. C duplicates sheets but never re-points them. D unions data *rows* together — it would duplicate/append records rather than replace the source.

> **Key point:** Re-pointing an entire workbook to a new source with the same schema = **Replace Data Source**.

Q5. ✅ **Answer: A** — A **left join** (Customers on the left) keeps every customer row once; non-matching customers get NULLs for the order fields, so you can identify customers with zero orders. B: inner join discards customers with no orders. C: keeps all *orders*, dropping customers without orders. D: a cross join pairs every row with every row.

> **Key point:** "Keep every X even when nothing matches in Y" → **left join with X on the left** (full outer joins both sides — use only when you must also keep unmatched Y rows).

Q6. ✅ **Answer: D** — Tableau's recommended default is a **relationship**: tables stay separate (no row duplication or unnecessary denormalization), Tableau matches rows automatically per-context, and many-to-many / mismatched-granularity scenarios are handled safely — ideal here (orders vs one-row-per-region-per-year targets). A: an outer join would create a physical, denormalized, duplicate-prone table. B: blending is a workaround, not the recommended structure. C: a union is wrong — the tables have different columns.

> **Key point:** Default = **relationships**. Choose joins only for same-granularity row-level merges; unions only for identical structure.

Q7. ✅ **Answer: B** — Identical-column files stacked end-to-end is **union** (Desktop: drag New Union / wildcard union in Prep's union step). A: relationships relate *different* tables in one source. C: blending combines *after aggregation* across sources. D: pivot changes rows↔columns, it doesn't append rows.

> **Key point:** "Same schema, append rows" → **union**. "Same detail level, merge columns" → **join**.

Q8. ✅ **Answer: A** — The **Data Interpreter** strips titles, merged cells, sub-total and footnote noise from Excel (also some CSV/PDF) files so Tableau can read clean rows. B: there is no "interpreter cache" option. C: pivot restructures columns→rows and does not delete junk rows. D: split breaks one string field into several — unrelated.

> **Key point:** Messy human-formatted Excel → **Data Interpreter**.

Q9. ✅ **Answer: C** — A **pivot** (columns to rows) reshapes the twelve month columns into Month/Value rows, producing a tidy long table you can chart over time. B: grouping combines *dimension members*. D: Data Interpreter cleans formatting, it does not reshape columns into rows.

> **Key point:** "Months/steps stored as several columns → turn them into rows" = **Pivot** (Rows-to-columns pivot is the reverse).

Q10. ✅ **Answer: D** — Values that differ in spelling but **sound alike** (Smith / Smyth / Smythe) are grouped with **Pronunciation** (Metaphone 3). B: Spelling (Levenshtein edit distance) matches near-identical spellings like `Smith` / `Smithh`. C: Common Characters (ngram fingerprint) matches values sharing characters, out of order. Manual Selection works, but the automatic, scenario-intended answer is Pronunciation.

> **Key point:** Sounds alike → **Pronunciation** • Spelled alike → **Spelling** • Shares characters (even out of order) → **Common Characters**.

Q11. ✅ **Answer: B** — The vendor must open the output **without Tableau**, so a universal text file (`.csv`) is correct. A: `.hyper` requires Tableau. C: `.twbx` is a Tableau workbook package. D: a published data source must be opened from Server/Cloud with Tableau.

> **Key point:** Prep output targets: `.hyper` (Tableau analysis), `.csv` (any tool / third parties), published data source (Server/Cloud users), database table.

Q12. ✅ **Answer: A** — A **data source filter** is applied at the data source level, so it affects every user and every workbook built on that source — the governance requirement. D: a context filter is set per worksheet by individual authors. C: table calc filters are per-view and only hide marks. B: same per-sheet limitation.

> **Key point:** "No user, anywhere, ever sees X in this source" → **data source filter**. (An extract filter would also exclude rows at extract build time, but the source-wide, always-on control the exam expects is the data source filter.)

Q13. ✅ **Answer: A, B** — Extracts are physically stored in Tableau's **columnar Hyper engine** (fast aggregations, offline use), and large append-style tables can use **full or incremental** refresh. C is false — extracts are snapshots; only live connections are real-time. D is false — extracts can be created from relational databases and cloud sources too.

> **Key point:** Multi-select discipline: look for absolute words ("only", "always") in options — they are usually the false statements.

Q14. ✅ **Answer: B → A → C → D** 1. **B** — Connect to the relational database. 2. **A** — Right-click the data source → Extract Data (or select the Extract radio button on the data source page). 3. **C** — Configure options: extract filters, aggregation, visible fields, row limit, incremental refresh. 4. **D** — Click **Extract**; Tableau creates and saves the `.hyper` file.

> **Key point:** Build-list items are graded all-or-nothing — verify the very first and very last steps first (connect first, save last), which usually anchors the whole sequence.

Q15. ✅ **Answer: C** — **Discrete** fields produce headers/labels; **continuous** fields produce axes. Converting `Profit Ratio` to continuous makes Tableau draw a numeric axis. A: converting to a dimension changes aggregation behavior, not blue/green. B: aliases rename member display values. D: sets define subsets of members.

> **Key point:** Blue = discrete = **headers**. Green = continuous = **axes**. Change with the right-click menu or by dragging the pill's left edge.

Section 2 — Explore and Analyze Data

Level of Detail (LOD) calculations

Q16. ✅ **Answer: B** — **FIXED LODs ignore regular (worksheet-level) dimension filters.** The calculation computes each country's total over its *entire* population — so the Berlin mark shows the total for **all German cities in the data**, including cities the filter hides. A is what a plain `SUM(Sales)` would show. C would require no fixed country: `{FIXED : SUM([Sales])}`. D is never how LODs respond to filters.

> **Key point:** FIXED LODs are computed *before* dimension filters — only **context, data source and extract filters** limit them. If you want a LOD to respect a filter, put that filter **in context**.

Q17. ✅ **Answer: D** — `{INCLUDE [Customer] : SUM([Sales])}` computes Sales at Region + Customer level and, displayed with **AVG**, returns the average of per-customer totals inside each region — exactly "average sales per customer within each region". B aggregates only to Region (region totals, no customer involvement). C excludes Region from the view level, collapsing to totals that ignore the regional grouping. D's `SUM` over fixed customer totals ignores the Region view entirely and double-counts logic; it never averages.

> **Key point:** "Average per within the current view level" → **INCLUDE** the detail dimension, then use AVG. INCLUDE respects view dimensions AND filters; FIXED ignores them.

Q18. ✅ **Answer: A** — On a Region + Country view, `{EXCLUDE [Country] : SUM([Sales])}` removes Country from the calculation level, so every mark shows its **Region-level total — repeated on each country row**. Typical use: quickly compare each country to its region (`SUM([Sales]) / {EXCLUDE [Country]: SUM([Sales])}` = % of region). B is plain `SUM(Sales)`. C would result from excluding *both* dims (`{FIXED : SUM(...)}`) rather than only Country. D is false — EXCLUDE is a valid, aggregatable LOD.

> **Key point:** EXCLUDE = "compute one level **up** from the view" (view minus the excluded dims). Classic exam use: totals-to-compare / top-level percentage denominators.

Q19. ✅ **Answer: C** — Nesting is evaluated from the **inside out**: `{FIXED [Region],[Customer] : SUM([Sales])}` = sales per region+customer; then the outer `{FIXED [Region] : MAX(… )}` picks, for each region, the maximum single-customer total — the best customer per region. A: inner LOD only fixes Region+Customer but nothing is returned per-customer first — this syntax computes MAX of region-level sums and does not retain per-customer depth as intended. B: `MAX([Sales])` without the customer grouping is the largest single *order*. D sums customer totals per region (= region total), it never finds the max.

> **Key point:** Nested LODs evaluate inside → out. Read the *innermost* expression first to decode what the outer aggregate does.

Q20. ✅ **Answer: B** — `{FIXED : SUM([Profit])}` — FIXED with **no** dimensions — computes one grand total and repeats it on every row regardless of view contents or filters. A varies with the dimensions in the view (and cannot be mixed with row-level fields without aggregation). C and D with no dimension are meaningless/unstable constructs whose results depend on the view level.

> **Key point:** "Same grand-total constant on every row/view" = **FIXED with an empty dimension declaration**. ({SUM([Profit])} is equivalent, but the exam-style written answer is `{FIXED : SUM([Profit])}`.)

Q21. ✅ **Answer: D** — The FIXED LOD partitions by Customer: Anna = 50+30 = **80**, Ben = 80, Cara = **20**. Every Cara row therefore carries 20. B is only her first order's value, C is the whole dataset's total, A is Anna's first order.

> **Key point:** To evaluate a FIXED LOD by hand: group rows by the declared dimension, aggregate, then attach the result back to every row of that group.

Q22. ✅ **Answer: A, B** — A is true: FIXED LODs are computed before dimension filters, so regular dimension filters don't change them. B is true: FIXED results don't depend on which dimensions are in the view (that's INCLUDE/EXCLUDE behavior). C is false — FIXED LODs **do** respect context filters. D is false — any aggregate (AVG, MIN, MAX, COUNTD…) is allowed.

> **Key point (memorize):** FIXED ignores: **dimension filters**. FIXED obeys: **context filters, data source filters, extract filters**.

Q23. ✅ **Answer: A** — The cohort date must attach to the customer regardless of the view, and it must survive other filters — exactly what `{FIXED [Customer] : MIN([Order Date])}` does. A's plain `MIN([Order Date])` re-computes at whatever level is in the view. C gets the *most recent* purchase. D's INCLUDE result re-aggregates back to the view level, so without Customer in the view it collapses to the overall minimum.

> **Key point:** Cohorts / first-event dates / "since their first purchase" → **FIXED per customer + MIN**.

Calculated fields — date, string, number, aggregate

Q24. ✅ **Answer: B** — `DATEDIFF('day', start, end)` counts **day boundaries crossed** between Jan 31 and Feb 2 = 1 (to Feb 1) + 1 (to Feb 2) = **2**. A: boundaries, not rows/length of interval-inclusive. C: off-by-one inclusive counting. D: would come from swapped arguments — always check the argument order `'part', start, end`.

> **Key point:** DATEDIFF is exclusive of both endpoints; negative results mean start is after end.

Q25. ✅ **Answer: C** — `DATENAME('month', [Order Date])` returns the month **name as a string** (e.g., "March"). A: DATEPART returns the number (3) — and it would sort correctly but display a digit. B: `MONTH()` is equivalent to DATEPART month — also a number. D: DATETRUNC returns a truncated *date* (2024-03-01), not a name.

> **Key point:** DATENAME = textual ("October") • DATEPART/MONTH = numeric (10) • DATETRUNC = first instant of the period (2024-10-01).

Q26. ✅ **Answer: A** — 2024 is a leap year; counting 90 days from Jan 1: 30 days to Jan 31, +29 days through Feb 29 (=59), +31 days through Mar 31 (=90). Result: **March 31, 2024**. B is simply "last day of February". C double-counts one day. D is off by one.

> **Key point:** For date arithmetic outputs, count day-boundaries carefully: Jan 1 + N days lands on Jan (N+1-th day of the offset table). Watch for leap years — the exam loves them.

Q27. ✅ **Answer: D** — DATEPARSE tokens must mirror the *string's* layout: `dd` = 17 (day), `MM` = 06 (month), `yyyy` = 2024 → `dd/MM/yyyy`. A would interpret 17 as a month — an invalid month, returning NULL. B expects a 4-digit year first. C expects dashes.

> **Key point:** DATEPARSE(pattern, string): the pattern describes the string you HAVE. In Tableau tokens, **uppercase MM = months, lowercase mm = minutes, dd = day-of-month**.

Q28. ✅ **Answer: B** — `RIGHT(string, howMany)` returns the **last n characters**: the last 3 of "Tableau" = **"eau"**. A (first 3 = LEFT), C (MID starting at position 3), D (characters 2–3).

> **Key point:** Argument order trap: `RIGHT("Tableau", 3)` ✓ — string first, count second. A reversed call like `RIGHT(3, "Tableau")` is invalid/misleading.

Q29. ✅ **Answer: C** — `INT()` **truncates toward zero**: −9.7 → **−9** (it chops the decimals). A (−10) would be rounding down / FLOOR behavior. B returns the unconverted float. D — type conversion does not error here.

> **Key point:** INT truncates toward zero (positive halves round… down: INT(9.7)=9). CEILING/FLOOR/ROUND behave differently — know all four.

Q30. ✅ **Answer: B, D** — `ZN([Sales])` converts NULL → 0 and passes other values through; `IFNULL([Sales], 0)` substitutes 0 when the expression is NULL. A: `ISNULL()` takes a single argument and returns true/false — this syntax is invalid. C: NULL + 0 = NULL, so this never fixes nulls. E: comparing to `= NULL` is always unknown in Tableau logic, so this returns Sales anyway.

> **Key point:** NULL toolkit: **ZN(x)** = x or 0 • **IFNULL(x, alt)** = x or alt • **ISNULL(x)** = true/false test • `= NULL` comparisons never work.

Table calculations

Q31. ✅ **Answer: C** — `RUNNING_SUM(SUM([Sales]))` accumulates: Jan 100 → Feb 150 → **Mar 350** → Apr 400. A is March's own value, D is April's running value, B is February's running value.

> **Key point:** Quick table calc "Running Total" = RUNNING_SUM. Read the table row by row and accumulate — the exam expects you to compute these by hand.

Q32. ✅ **Answer: C, D** — `LOOKUP()` and `INDEX()` are **table calculation** functions (they operate on the values already laid out in the view). A: ZN is a NULL-handling data function. B: ATTR is an *aggregate*. E: SUM is a plain *aggregate*.

> **Key point:** You cannot mix plain aggregates and table-calc functions in one expression (e.g., `SUM(INDEX())` fails). Learn the three families: aggregates (SUM/AVG/COUNTD/ATTR…), table calc (RUNNING_SUM/WINDOW_*/LOOKUP/INDEX/RANK/FIRST/SIZE…), row-level (data) functions.

Q33. ✅ **Answer: C** — `LOOKUP(SUM([Sales]), -1)` returns the value **one row back** within the partition; for Feb that is Jan's value = **100**. A is Feb's own value; B is March's; D is what it returns on the **first** mark (no previous row exists).

> **Key point:** Positive offsets look forward, negative look backward, offset 0 = current mark; the first/last row returns NULL when it runs off the edge.

Q34. ✅ **Answer: D** — `RANK_DENSE()` gives ties the same (dense) rank and uses **no gaps**: 1, 2, 2, 3. A: `RANK()` = competition ranking with gaps: 1, 2, 2, **4**. B: `RANK_UNIQUE()` numbers all rows uniquely: 1, 2, 3, 4. C: `RANK_MODIFIED()` = 1, 3, 3, 4.

> **Key point:** Memorize the tie patterns for [100, 80, 80, 50]: RANK → 1,2,2,4 • RANK_DENSE → 1,2,2,3 • RANK_MODIFIED → 1,3,3,4 • RANK_UNIQUE → 1,2,3,4.

Q35. ✅ **Answer: B** — `WINDOW_AVG(SUM([Sales]), -2, 0)` averages the current and two previous marks. For April: (20 + 30 + 40) / 3 = **30**. A (25) is the average of *all four* months — that's what you get if window bounds were `-3, 0` or unbounded. C is April's raw value. D is January's edge-truncated value.

> **Key point:** Windows **truncate at the partition edge** — January averages only itself. Also know WINDOW_SUM and the quick calc "Moving Average" (e.g., "Previous 2 Values" computes exactly this).

Q36. ✅ **Answer: A** — Percent of Total is computed **before** a table calc filter runs, and a table calc filter only *hides* marks — it does not remove their values from the table calc denominator. West therefore keeps its original **30%**. B (100%) would appear only if West were the *only* mark *and* you recomputed with a dimension filter instead. C / D have no basis.

> **Key point (top exam trap):** Table calc filters are applied **last** — they hide marks but don't change other marks' table calculations. Dimension filters, applied earlier, DO change the % of total denominators.

Q37. ✅ **Answer: C** — A Top-N filter normally ranks across the *whole* data source. Converting Category to a **context filter** makes Tableau materialize the subset first, so the Top 10 is then computed **within** Furniture. A adds unnecessary complexity over a plain table-calc rank and still needs context to scope. B is an axis feature, unrelated to Top N. D (data source filter) restricts data for everyone — but "Top 10 within a *user-selected* category" requires the interactive context approach.

> **Key point:** "Top N **of a subset** / within a selection" → **context filter** (pill turns gray-blue) + Top N filter. Also remember context filters improve performance on large extracts.

Q38. ✅ **Answer: A, C, D** — Parameters may be used **in calculated fields** (what-if, measure swap), **with filters** (dynamic Top N), and **with reference lines** (user-adjustable targets); they can even define bin sizes. B is false — a parameter holds exactly **one** value at a time (that's a set / multi-select filter's job). E is false — the *selected value* doesn't auto-update with data; only the parameter's *list of allowed values* can refresh dynamically from a field.

> **Key point:** Parameter = one workbook-level value the user controls. Set = a list of members. Filter = restricts data. Know which tool answers which "interactivity" scenario.

Q39. ✅ **Answer: B, C** — B: a **dynamic set** built from a condition (e.g., Top 10 by SUM(Sales)) re-evaluates as data refreshes. C: a **combined set** joins exactly two sets — and the only supported combination is the **intersection** (with in/out options). A is false (that describes a *fixed* set, not sets in general). D is false — sets are built from dimension members (conditions may use aggregates).

> **Key point:** Fixed set = manually chosen members (constant). Dynamic set = condition-based, auto-updating. Combined set = intersection of two sets only.

Q40. ✅ **Answer: B** — `MAKEPOINT([Latitude], [Longitude])` builds a spatial point object (works with unprojected lat/long) that maps like a geographic field. A: MAKELINE creates a line between two points. C reverses the argument order (longitude first is wrong for MAKEPOINT). D: DISTANCE measures kilometers/miles *between* two points.

> **Key point:** Spatial function trio: **MAKEPOINT**(lat, long), **MAKELINE**(start pt, end pt), **BUFFER**(point, km). DISTANCE needs two points. Latitude comes first.

Q41. ✅ **Answer: A** — A fixed target of 50,000 is a **constant reference line** (Analytics pane → Reference Line → Constant). B: average line computes the *mean of the marks*, which moves with filters — wrong for a fixed target. C: trend lines model direction. D: distribution bands show percentiles/standard-deviation spread.

> **Key point:** Fixed target → **constant reference line**. Computed (average, sum, max, min) → reference line with computed value, scoped to Table/Pane/Cell as needed.

Q42. ✅ **Answer: C → F → A → D → E → B** 1. **C** Extract filters (when the extract is built) 2. **F** Data source filters (source level, all users) 3. **A** Context filters 4. **D** Dimension filters 5. **E** Measure filters 6. **B** Table calculation filters (last — they only hide/transform what's left)

> **Key point:** This order explains every "why doesn't my filter change my LOD / % of total?" exam question: FIXED LODs sit after context filters but before dimension filters; table calc filters run last.

Section 3 — Create Content

Q43. ✅ **Answer: B** — Pie marks require a **continuous measure on Angle** and dimensions on **Color** and **Label**: Genre → Color (and Label), Revenue → Angle. A: Revenue on Color splits slices by revenue values, and Genre on Angle is invalid (needs a numeric/continuous field). C: Genre (a dimension) on Size is invalid for pie marks. D: Revenue on Size is correct only for the angle slot misuse — with Genre on Detail you just get multiple zero-width slices; the slices split evenly with revenue as labels.

> **Key point (official practice-question style):** PIE = **dimension on Color/Label, measure on Angle**. (Angle is the same slot Size occupies on other mark types.)

Q44. ✅ **Answer: B → A → D → C** 1. **B** — Right-click the measure → **Create → Bins**, set the bin size. 2. **A** — Drag the bin field to **Columns** (discrete headers). 3. **D** — Drag the measure to **Rows**. 4. **C** — Change its aggregation to **Count (CNT)** — the bar heights are row counts per bin.

> **Key point:** Histogram = **bins of a continuous measure + COUNT**. (Show Me's histogram does all four steps in one click — know the manual path anyway.)

Q45. ✅ **Answer: C → B → D → A → E** 1. **C** — Drag the first measure (Sales) to Rows. 2. **B** — Drag the second measure (Profit Ratio) to Rows → two stacked charts. 3. **D** — Right-click the second pill → **Dual-Axis** (or drag it onto the right edge of the view). 4. **A** — Right-click the right axis → **Synchronize Axis** (Tableau does NOT do this automatically). 5. **E** — Set each measure's mark type (Bar + Line) via the Marks card.

> **Key point:** The #1 dual-axis exam trap: **synchronize the axes yourself**, and only trust comparisons after you do.

Q46. ✅ **Answer: D** — A **box plot** shows median, quartiles, whiskers and flag points outside 1.5×IQR as outliers — exactly the request. A: area charts show cumulative/trend shape. B: Gantt shows duration/schedule. C: highlight tables color cells by value but don't compute distributions.

> **Key point:** Distribution questions: counts/shape → **histogram**; median/quartiles/outliers → **box plot**; relationship between two measures → **scatter plot**.

Q47. ✅ **Answer: C** — Right-click the Category pill → **Sort → By Field → Sales (Sum), Descending**. This ties the sort to the measure everywhere in the view. B: moving a pill re-orders shelves, not bar order. A: continuity affects axis/headers. D: INDEX is a table calc used for row numbers, not sorting.

> **Key point:** "Sort by value, not alphabetically" → right-click the dimension → Sort → **By field** (or click the toolbar sort icons which toggle alphabetical ⇄ field-based for the current view).

Q48. ✅ **Answer: A** — **Layout containers** (horizontal/vertical) keep their children aligned and let them resize proportionally together as the dashboard flexes. B: floating objects hold pixel positions and don't reflow. C: a text object is just decoration. D: blank objects only add spacing.

> **Key point:** Tiled = auto-stack; floating = pixel-pinned; **containers = disciplined layout** that survives resizing (also used for show/hide zones).

Q49. ✅ **Answer: D** — **Dynamic zone visibility**: create a calculation driven by a parameter (e.g., `IF [pShowDetail] = "Show" THEN "Shown" ELSE "Hidden" END`), then on the Layout pane set **"Control visibility using value"**. A/B/C are real features but cannot show/hide a sheet based on a parameter.

> **Key point:** Dynamic zone visibility works on the LAYOUT pane with a boolean/string calculation using parameters or set-driven logic — a newer, heavily tested interactivity topic.

Q50. ✅ **Answer: B** — **Device Preview** renders the dashboard in phone/tablet presets; you then generate a **phone layout** and rearrange/remove items for small screens. A/D/C don't address responsive design.

> **Key point:** "Responsive design on Tableau" = dashboard size (fixed/range/automatic) + **Device Preview** → **device-specific layouts**.

Q51. ✅ **Answer: D → A → B → C** 1. **D** — Build the two worksheets and assemble them on a dashboard. 2. **A** — Dashboard menu → Actions → Add Action → Filter. 3. **B** — Set the Source sheet (map), Target sheet(s) (detail chart), and Run On: **Select**. 4. **C** — Save; click a mark and confirm the target filters.

Options for clearing/selection ("All values", "Show all values" on selection clearing) are configured in the same dialog.

> **Key point:** Filter actions: source → target, run on **Select / Hover / Menu**. Hover can annoy users; Select is the default choice.

Q52. ✅ **Answer: C** — A **URL action** can build the web address dynamically by **inserting field values** from the clicked mark (e.g., a Product ID) into the URL. A/B: an image object or a Web Page object shows a *fixed* page/asset — they cannot react to the clicked mark. D: navigation buttons move between sheets, they don't pass mark context.

> **Key point:** URL action = mark context → dynamic web address (supports field insert, e.g. `https://supplier.com/product/<Product ID>`).

Q53. ✅ **Answer: A** — A **set action** changes set membership when marks are clicked/hovered; other sheets using the set (in/out coloring, calcs, filters) update automatically. B: highlight only emphasizes marks. C: filter actions hide marks on *other sheets* but can't drive set-based logic. D: parameter actions set a parameter value — no set membership involved.

> **Key point:** Set action = click → set membership → anything driven by the set updates (very testable: "click a customer to drill into their details").

Q54. ✅ **Answer: D** — Custom color palettes are defined in the **`Preferences.tps`** file inside the **My Tableau Repository** folder using `<color-palette type="regular">` XML entries; the palette then appears in every workbook's color picker. A: workbook XML isn't the mechanism. B: .hyper is data. C: there is no in-app palette editor.

> **Key point:** Classic question: **Preferences.tps + My Tableau Repository** = custom palettes. Custom shapes work the same way via the Shapes folder.

Q55. ✅ **Answer: B** — **Viz in Tooltip**: in the tooltip editor choose **Insert → Sheets** → the mini sheet; optionally "Filter all fields" so it scopes to the hovered customer. A: annotations are static notes. C: highlight actions emphasize but don't render a chart. D: reference bands add statistical bands within a view.

> **Key point:** Hover-driven mini charts = **Viz in Tooltip** (set the sheet's size, use "Filter all fields" for per-mark context).

Section 4 — Publish and Manage Content on Tableau Server and Tableau Cloud

Q56. ✅ **Answer: C** — A `.twbx` bundles the workbook with **local file-based data sources, extracts and images**, letting recipients open it without the originals — but it cannot embed data behind **live** connections (the recipient still needs database access). A is false (live data is never packaged). B is false (`.twbx` opens in Desktop/Reader). D is false — it contains real workbooks, not pictures.

> **Key point:** Need a DB-connected workbook to keep live data? Publish to Server/Cloud with **embedded credentials + "Allow refresh"** instead of relying on .twbx.

Q57. ✅ **Answer: A** — An **incremental refresh** adds only new rows since the last refresh (identified by a date/timestamp column) — hours of work become minutes. B: a data source filter permanently excludes data; it doesn't append new rows. C: live would bypass the extract entirely but reintroduces the slow-query problem. D: subscriptions email view images.

> **Key point:** "Huge append-only table; refresh only new rows" → **incremental extract refresh**.

Q58. ✅ **Answer: D** — **Alerts** are data-driven: set on a published worksheet, they email you when a threshold is crossed (sales < 10,000) — condition-based, immediate. A: subscriptions email a **scheduled snapshot** at fixed times, regardless of values. B: custom views save personal state. C: extract refreshes update data.

> **Key point:** "Email me WHEN the data condition occurs" → **alert**. "Email me EVERY day at 8 AM" → **subscription**.

Q59. ✅ **Answer: B** — **Custom Views** are per-user snapshots of the current filters/sort/selection on a published workbook; you can save, name, share via link, make it your default — nothing changes for other users. A: republishing creates content and requires rights. C/D: those filters affect the data source for everyone.

> **Key point:** Personalization of published content without affecting others = **custom views** (Viewer-friendly).

Q60. ✅ **Answer: A, B** — A: Viewers can see, filter, and interact with published dashboards (subscriptions/custom views too) but cannot edit. B: Creators can connect to new data, author content (web or Desktop), and publish. C is false — publishing flows and connecting to new data require the **Creator** role (some Explorer variants may edit existing workbooks, but not publish new flows/data). D is false — row-level visibility is controlled by data policies/row-level security and permissions, **not** by site roles.

> **Key point:** Roles ladder: **Viewer** (interact) → **Explorer** (edit existing, possibly publish views) → **Creator** (connect data, author, publish, flows). Permissions ≠ site roles ≠ data policies.

📊 Score yourself

Domain	Questions	Correct	Weight target (exam)
1. Connect & Transform	Q1–15 (15)	___ / 15	24%
2. Explore & Analyze	Q16–42 (27)	___ / 27	41%
3. Create Content	Q43–55 (13)	___ / 13	26%
4. Publish & Manage	Q56–60 (5)	___ / 5	9%
TOTAL	60	___ / 60	**Pass = 39 (65%)**

Interpreting your result

- **55–60:** exam-ready — take the real exam.
- **45–54:** nearly there — re-read every *Key point* line, re-test missed areas after 3 days.
- **39–44:** borderline — drill Domain 2 again (it's 41% of the real exam) and retake this paper.
- **< 39:** target your weakest domain from the table above; redo the LOD (Q16–23) and table calc (Q31–36) blocks until every "Key point" is automatic.

Last review checklist before exam day (the ten things most likely to earn/drop points)

1. FIXED ignores dimension filters; obeys context/data-source/extract filters.
2. INCLUDE = view + extra detail; EXCLUDE = view − a level; nested LODs read inside→out.
3. Table calc filters run LAST (they hide marks; % of total stays the same).
4. Tie ranks: RANK 1,2,2,4 • RANK_DENSE 1,2,2,3 • RANK_MODIFIED 1,3,3,4.
5. INT truncates toward zero; DATENAME = text, DATEPART = number; DATEPARSE pattern mirrors the string.
6. Pie = dimension on Color/Label + measure on Angle; dual-axis = synchronize yourself.
7. Live vs extract vs incremental refresh decision phrases.
8. Join vs relationship vs union vs blend; Prep Group Values (Pronunciation/Spelling/Common Characters).
9. Parameters hold ONE value; used in calcs, filters, reference lines. Alerts are conditional; subscriptions are scheduled.
10. Read "Select TWO/THREE" carefully — no partial credit; never leave anything unanswered.